



Understanding Memory Areas in a C Program

What happens when a C program is loaded into memory? Where are the different types of variables allocated? Let's look at some of these interesting 'under the hood' details.

Ravikiran from Hyderabad (a regular reader of my *Joy of Programming* column) asked me this: "Why do we need two data sections—initialised and un-initialised? If I initialise a static or a global variable with zero, where will it be stored? Since the scopes of global and static variables are different, why are they stored in the same data section?" These queries prompted me to write this article, which should interest any assembly language/C/C++ programmer.

Four important segments

Let us first understand the memory layout of a C program (which is compiled to an executable and loaded into the memory for execution). There are four main segments in a C program: data, code, stack and heap segments.

Global and function static variables are allocated in the data segment. The C compiler converts executable statements in a C program—such as `printf("hello world");`—into machine code; they are loaded in the code segment. When the program executes, function calls are made. Executing each function requires allocation of memory, as if in a frame, to store different information like the return pointer, local variables, etc. Since this allocation is done in the stack, these are known as 'stack

frames'. When we do dynamic memory allocation, such as the use of the `malloc` function, memory is allocated in the heap area.

The data and text areas are of fixed size. When a program is compiled, at that point itself, the sizes required for these segments are fixed and known—hence, they are also known as static segments. The sizes of the stack and heap areas are not known when the program gets compiled. Also, it is possible to change/configure the size of these areas (i.e., increase or decrease the size of these segments); so these areas are known as dynamic segments.

Let us look at each of these segments in detail now. For starters, we'll explore an example program and a tool to find out where the variables get allocated later.

Data segment

The data segment is to hold the value of those variables that need to be available throughout the lifetime of the program. So, it is obvious that global variables should be allocated in the data segment. How about local variables declared as static? Yes, they are also allocated in the data area because their values should be available across function calls. If they are allocated in the stack frame itself, they will get destroyed once the function returns. The only option is to allocate them in a global